

Relatório do Trabalho 04

André Malmsteen Oliveira Amorim, Benjamim Isaac Ribeiro Lima,
Giovanna Bembom da Silva Bandeira, Mariana Ramos André Simões

¹Instituto de Computação (IComp) – Universidade Federal do Amazonas (UFAM)
Av. Gen. Rodrigo Octávio 6200, Coroado I – 69080-900
Campus Universitário – Setor Norte – Manaus – AM – Brasil

{andre.mlmst, benjamim.lima}@icomp.ufam.edu.br,

{giovanna.bembom, mariana.simoes}@icomp.ufam.edu.br

1. Introdução

Este trabalho foi desenvolvido utilizando um computador pessoal com configurações adequadas para execução de experimentos computacionais e análise de desempenho de algoritmos. O dispositivo utilizado é um *notebook* Lenovo IdeaPad 3 15ITL6, equipado com processador Intel Core i5-1135G7 de 11ª geração, operando a uma frequência base de 2.40 GHz.

A máquina possui 8 GB de memória RAM com velocidade de 3200 MT/s, sendo aproximadamente 7,79 GB utilizáveis pelo sistema. O armazenamento total disponível é de 238 GB, dos quais cerca de 111 GB encontram-se em uso no momento da realização dos experimentos. Em relação ao processamento gráfico, o dispositivo conta com uma GPU integrada Intel Iris Xe Graphics.

O sistema operacional utilizado é o *Windows 11 Home Single Language*, versão 25H2, com arquitetura de 64 bits, garantindo compatibilidade com aplicações modernas e suporte a execução eficiente dos programas desenvolvidos.

Essas especificações fornecem uma base computacional suficiente para a implementação, execução e análise de algoritmos, permitindo a coleta de métricas de desempenho de forma confiável ao longo dos experimentos realizados neste trabalho.

Os códigos implementados estão disponíveis no GitHub, podendo ser acessados pelo seguinte *link*: Repositório Trabalho4_AEDII (https://github.com/giovannabembom12/Trabalho4_AEDII).

2. Criação de Grafos com Graus de Conectividade Variados (Questão 1)

O objetivo da Questão 1 do trabalho prático é avaliar a rotina de geração de grafos conexos aleatórios. O algoritmo utilizado na questão segue tipicamente três etapas:

1. **Criação dos vértices:** são alocados n vértices, numerados de 0 a $n - 1$.
2. **Garantia de conectividade (árvore geradora):** para assegurar que o grafo resultante seja *conexo* (conforme o próprio nome da função indica), o algoritmo primeiro conecta todos os vértices através de uma árvore geradora aleatória, consumindo exatamente $n - 1$ arestas – o número mínimo necessário para conectar n vértices sem ciclos.
3. **Inserção de arestas aleatórias adicionais:** em seguida, arestas aleatórias (evitando laços e duplicatas) são inseridas até que o número total de arestas do grafo atinja o valor alvo determinado pelo percentual de conectividade solicitado.

O trecho de código responsável pela geração das saídas analisadas é reproduzido a seguir:

Listing 1: Função `questao01`, responsável por gerar os grafos com diferentes números de vértices e percentuais de conectividade.

```
1 static void questao01(void) {
2     int vertices[] = {5,10,20,50,100};
3     double percentagens[] = {0.25, 0.50, 0.75, 1.00};
4     printf("\n-- Questao 1: Criacao de grafos com graus de "
5           "conectividade variados --\n");
6     for(int i=0; i<5; i++){           // num de grafos criados
7         int n = vertices[i];
8         for(int j=0; j<4; j++){       // passagem pelas
9             percentagens
10            double percentagem = percentagens[j];
11            Grafo *grafo = grafo_gerar_conexo(n, percentagem);
12            printf("Grafo criado com %d vertices e conectividade
13                  "
14                  "de %.2f%%\n", n, percentagem);
15            grafo_imprimir_lista(grafo);
16            grafo_destruir(grafo);
17        }
18    }
19 }
```

O programa itera sobre 5 tamanhos de vértices e, para cada tamanho, sobre 4 percentuais de conectividade, totalizando 20 grafos distintos gerados e impressos como listas de adjacência.

2.1. Visualização dos Grafos Gerados

Nesta seção são apresentadas as representações gráficas (diagramas de nós e arestas) reconstruídas a partir das listas de adjacência. Como verificação de consistência, o número de vértices e arestas foi comparado com os valores impressos na saída original.

Para grafos com $n = 50$ e $n = 100$ vértices, a densidade de arestas torna a representação em diagrama de nós pouco legível visualmente; por esse motivo, tais casos são analisados apenas de forma quantitativa, sendo o eixo qualitativo (visual) restrito aos grafos com $n \in \{5, 10, 20\}$, para os quais a inspeção visual da conectividade é mais proveitosa.

2.2. Grafos com $n = 5$ vértices

A Figura 1 mostra os quatro grafos gerados com 5 vértices, para os percentuais de 25%, 50%, 75% e 100% de conectividade. É possível observar visualmente a transição de uma estrutura próxima de uma árvore (esparsa, porém conexa) até o grafo completo K_5 .

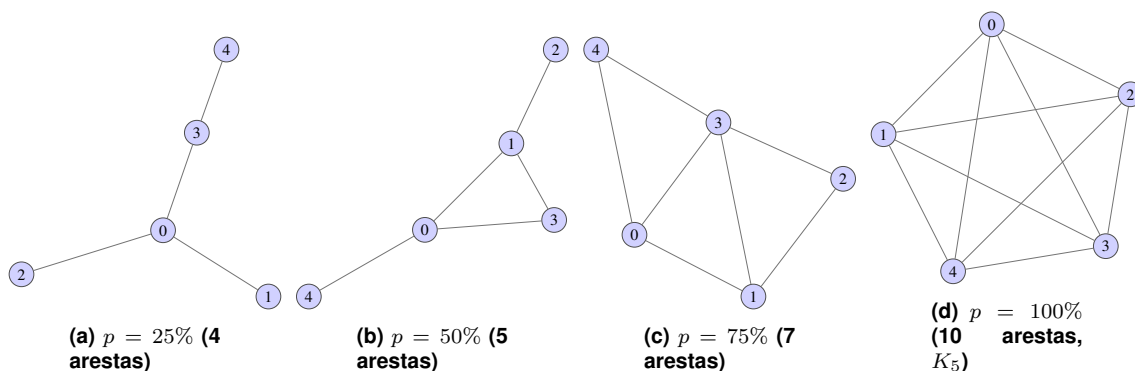


Figura 1: Grafos gerados com $n = 5$ vértices para os quatro percentuais de conectividade avaliados.

2.3. Grafos com $n = 10$ vértices

De forma análoga, a Figura 2 apresenta os grafos com 10 vértices. Nota-se que, mesmo com $p = 25\%$, o grafo já apresenta uma estrutura com alguns vértices de grau mais elevado atuando como conectores entre componentes que, de outra forma, estariam isolados – um efeito direto da árvore geradora aleatória subjacente.

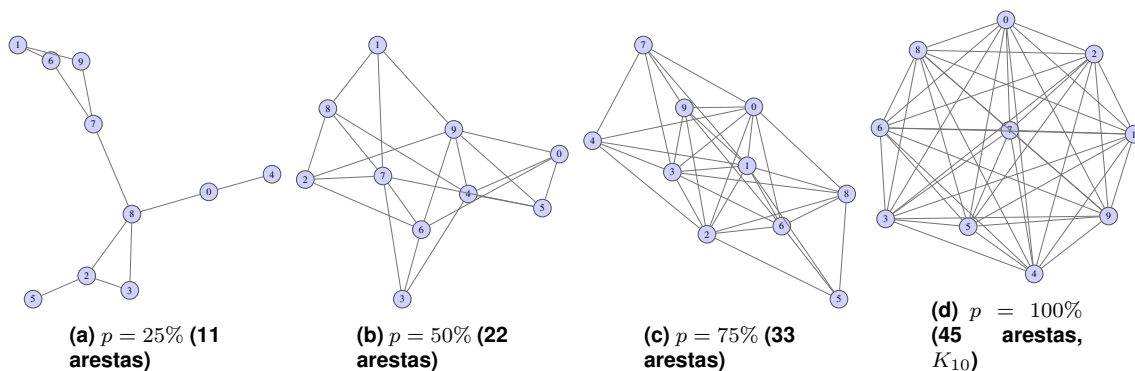


Figura 2: Grafos gerados com $n = 10$ vértices para os quatro percentuais de conectividade avaliados.

2.4. Grafos com $n = 20$ vértices

Por fim, a Figura 3 exibe os grafos com 20 vértices para os percentuais de 25% e 50% de conectividade, ilustrando como o aumento da densidade rapidamente satura visualmente o espaço do diagrama, aproximando-se de uma nuvem densa de conexões.

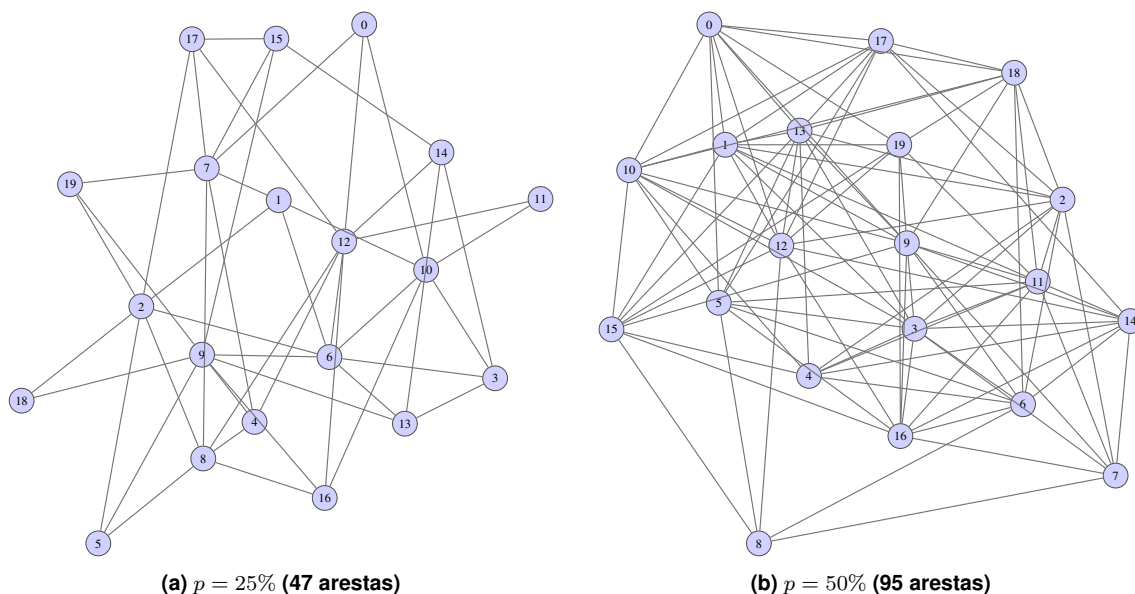


Figura 3: Grafos gerados com $n = 20$ vértices para os percentuais de 25% e 50% de conectividade.

As visualizações apresentadas na Seção 4 (Figuras 1, 2 e 3) ilustram de forma qualitativa a transição de estruturas esparsas (próximas de árvores) para estruturas densas (grafos completos), reforçando visualmente os resultados quantitativos discutidos ao longo deste relatório.

3. Busca em Largura - BFS (Questão 2)

O TAD *Grafo* representa o grafo por meio de um vetor de listas encadeadas (uma lista de adjacência por vértice), de forma análoga aos TADs de árvore e vetor já utilizados em trabalhos anteriores da disciplina. A função *gerarGrafoConexo* constrói grafos de teste em duas etapas: (i) gera-se uma árvore geradora aleatória, conectando cada vértice a um vértice já inserido escolhido ao acaso, o que garante a conexidade do grafo com o número mínimo de arestas ($n - 1$); (ii) arestas adicionais são inseridas aleatoriamente até que a fração desejada do total de arestas possíveis ($\binom{n}{2}$) seja atingida, de acordo com o grau de conectividade solicitado (25%, 50%, 75% ou 100%).

A função *buscaEmLargura* implementa o algoritmo de BFS com o auxílio de uma fila (implementada em vetor, já que o tamanho máximo de vértices na fila é conhecida) e de vetores auxiliares *pai* e *nivel*, que permitem reconstruir a árvore de busca resultante e a distância (em número de arestas) de cada vértice até a origem. O tempo de execução foi medido com a função *tempoAtual* do módulo *estatisticas*, que utiliza `clock_gettime(CLOCK_MONOTONIC, ...)` para os resultados. Para cada combinação de (quantidade de vértices, grau de conectividade), o grafo foi gerado uma única vez e a BFS foi executada 10 vezes a partir de vértices iniciais escolhidos aleatoriamente a cada repetição, registrando-se o tempo individual de cada busca, a média e o desvio padrão (calculados pelas funções *media* e *desvioPadrao*, também do módulo *estatisticas*).

3.1. Exemplo

Para permitir a inspeção visual do funcionamento do algoritmo, a Figura 4 mostra um grafo de teste pequeno, gerado com $n = 5$ vértices e grau de conectividade de 25%, cuja

lista de adjacência resultante foi:

Vértice	Adjacências
0	2, 1
1	3, 4, 0
2	0
3	1
4	1

A Figura 5 mostra a árvore de busca em largura resultante a partir do vértice 0, com a ordem de visitação 0, 2, 1, 3, 4.

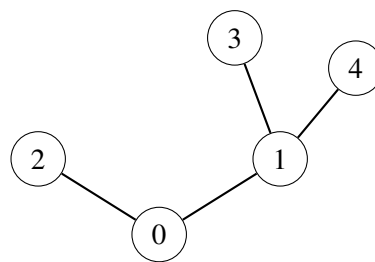


Figura 4: Grafo de teste com 5 vértices e grau de conectividade de 25% (4 arestas).

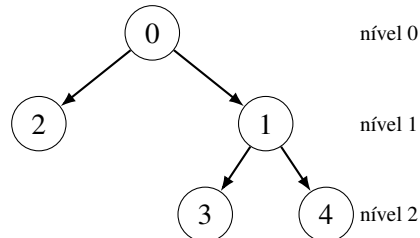


Figura 5: Árvore de busca em largura obtida a partir do vértice 0.

Esse mesmo procedimento foi repetido para grafos com 10, 20, 50 e 100 vértices, sempre variando o grau de conectividade entre 25% e 100%, conforme detalhado a seguir.

3.2. Resultados

A Tabela 1 apresenta o tempo médio (em segundos, média de 10 execuções) da BFS para cada combinação de quantidade de vértices e grau de conectividade testada.

A Figura 6 apresenta os mesmos dados em forma de gráfico, com o eixo do tempo em escala logarítmica devido à ampla faixa de valores observada (de $4,8 \times 10^{-6}$ s a $4,6 \times 10^{-4}$ s).

Tabela 1: Tempo médio de execução da BFS (10 buscas) em segundos.

2*Vértices (n)	Grau de conectividade			
	25%	50%	75%	100%
5	9,71E-06	7,28E-06	7,53E-06	4,80E-06
10	2,22E-05	9,70E-06	5,39E-06	5,93E-06
20	2,17E-05	1,39E-05	1,23E-05	1,26E-05
50	2,03E-05	2,95E-05	1,25E-04	6,10E-05
100	9,11E-05	2,80E-04	2,29E-04	4,59E-04

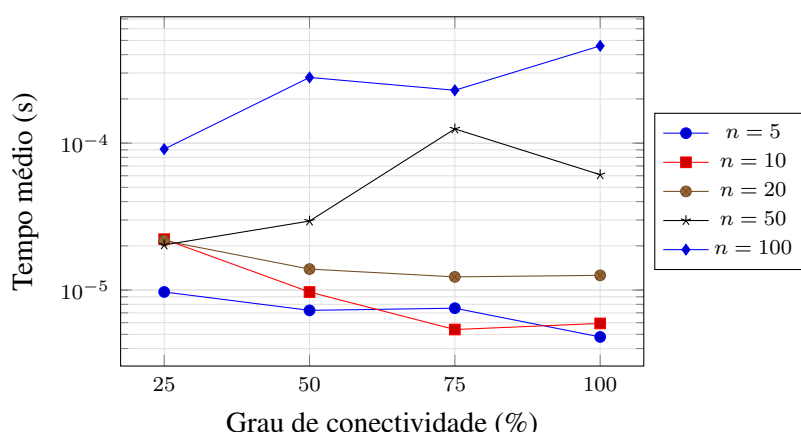


Figura 6: Tempo médio de BFS em função do grau de conectividade, para diferentes quantidades de vértices.

3.3. Relação entre grau de conectividade e tempo de execução.

Como a BFS visita cada vértice uma única vez e percorre cada aresta no máximo duas vezes, sua complexidade de tempo é $O(V + E)$. Para um grafo com n vértices, o número de arestas cresce de forma aproximadamente linear com o grau de conectividade (de $n - 1$ arestas, no caso conexo mínimo, até $\binom{n}{2}$ arestas no grafo completo). Espera-se, portanto, que o tempo de execução cresça com o grau de conectividade, efeito que fica evidente nos grafos maiores ($n = 50$ e $n = 100$ na Tabela 1), nos quais o tempo médio aumenta em até uma ordem de grandeza entre 25% e 100% de conectividade. Para os grafos pequenos ($n = 5, 10, 20$), entretanto, essa relação não é clara e chega a ser inversa em alguns pontos (por exemplo, $n = 10$ tem tempo *menor* a 100% do que a 25% de conectividade). Isso ocorre porque, nesses tamanhos, o tempo total de execução da BFS (da ordem de microssegundos) é comparável à resolução do temporizador do sistema e ao custo fixo de chamadas de função, alocação de memória (*malloc* da fila e dos vetores auxiliares) e efeitos de *cache*: nessa faixa, o ruído de medição domina o sinal relacionado a E . O mesmo fenômeno explica os valores isolados muito acima da média observados em algumas buscas individuais (por exemplo, a busca 7 do grafo com $n = 10$ e 25% de conectividade, que levou $1,55 \times 10^{-4}$ s contra uma média de $2,5 \times 10^{-5}$ s nas demais repetições daquele grafo), tais picos são atribuíveis a interrupções do sistema operacional (trocas de contexto, *cache misses*) e não ao algoritmo em si, o que justifica o uso da média (e do desvio padrão) sobre 10 repetições em vez de uma única medição.

Em resumo, há relação entre grau de conectividade e tempo de execução da BFS,

mas ela só se torna estatisticamente visível a partir de um número de vértices grande o suficiente para que o tempo de execução supere o ruído de medição do sistema. No conjunto de testes realizado, essa condição ficou clara a partir de $n = 50$.

3.4. Comparação com a Busca em Profundidade (Questão 3)

A Tabela 2 e a Figura 7 comparam o tempo médio da BFS (Seção 3.2) com o tempo médio da DFS (Questão 3), fixando o grau de conectividade em 50% e variando o número de vértices.

Tabela 2: Tempo médio (s) de BFS e DFS a 50% de conectividade.

Vértices (n)	BFS	DFS
5	7,28E-06	9,50E-04
10	9,70E-06	1,78E-03
20	1,39E-05	3,45E-03
50	2,95E-05	9,56E-03
100	2,80E-04	1,95E-02

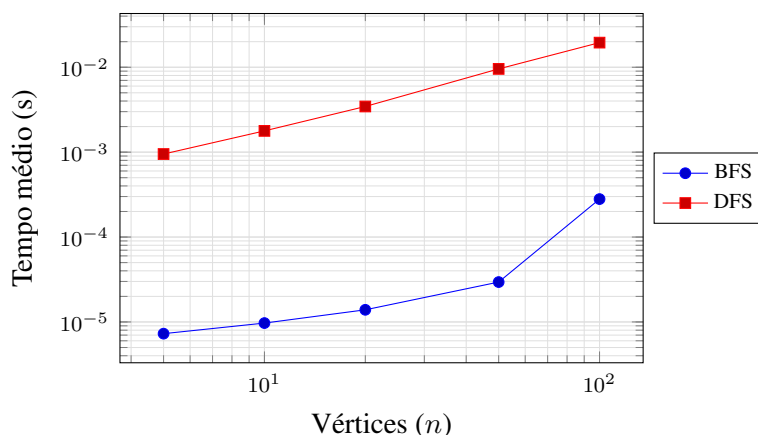


Figura 7: Crescimento do tempo médio de BFS e DFS com o número de vértices (grau de conectividade fixo em 50%, escala log-log).

Dois pontos chamam a atenção nessa comparação. Primeiro, em termos absolutos, a DFS (Questão 3) apresentou tempos médios de duas a três ordens de grandeza maiores que os da BFS para os mesmos grafos, mesmo ambas tendo complexidade assintótica $O(V + E)$. Isso é compatível com uma diferença de constantes de implementação: a BFS aqui usa uma fila auxiliar em vetor e laços iterativos simples, enquanto uma implementação recursiva de DFS paga, a cada vértice visitado, o custo de uma nova chamada de função (empilhamento de quadro de pilha, salvamento de registradores, retorno), o que é sensivelmente mais caro do que uma iteração de laço – especialmente em grafos com muitos vértices, nos quais a recursão pode atingir profundidades consideráveis. Segundo, observando a taxa de crescimento (inclinação das retas na Figura 7, em escala log-log), nota-se que o tempo da DFS cresce de forma mais regular e aproximadamente linear com n (compatível com $O(V + E)$ dominado pela constante por vértice), ao passo que o tempo da BFS tem crescimento mais irregular em n pequeno (efeito do ruído de medição já discutido) e só passa a acompanhar claramente o crescimento esperado a partir de $n = 50$.

Do ponto de vista assintótico, BFS e DFS utilizam estruturas auxiliares de tamanho proporcional a V (vetores de `pai/nivel/visitados` e, respectivamente, uma fila ou uma pilha/recursão), de modo que ambas têm complexidade de espaço $O(V)$ além do próprio grafo. Não se espera, portanto, diferença assintótica relevante de uso de memória entre as duas buscas. A diferença prática está na natureza dessa memória auxiliar: a fila da BFS é alocada explicitamente em um único bloco de tamanho n , com custo previsível, enquanto a pilha de chamadas usada por uma DFS recursiva cresce dinamicamente a cada chamada e pode, em grafos muito profundos (por exemplo, grafos pouco conectados e com formato de "caminho"), aproximar-se do limite de pilha do sistema operacional – um risco que a BFS, por não usar recursão, não apresenta. Essa é uma diferença qualitativa relevante mesmo quando o consumo total de memória (em bytes) é semelhante.

3.5. Conclusão

Os experimentos confirmam o comportamento esperado da Busca em Largura: sua complexidade $O(V + E)$ se traduz em tempos de execução crescentes com o grau de conectividade, efeito claramente observável a partir de grafos de médio/grande porte ($n \geq 50$ neste estudo). Em grafos pequenos, o custo fixo de execução (alocação de memória, chamadas de função, resolução do temporizador) supera o efeito do número de arestas, tornando a relação entre conectividade e tempo estatisticamente pouco confiável nessa faixa – reforçando a importância de se repetir cada medição (10 vezes, neste trabalho) e reportar média e desvio padrão em vez de uma única execução. Frente à Busca em Profundidade (Questão 3), a BFS aqui implementada mostrou tempos consistentemente menores, atribuídos à ausência de sobrecarga de chamadas recursivas, ainda que ambos os algoritmos compartilhem a mesma complexidade assintótica de tempo e de espaço.

4. Busca em Profundidade - DFS (Questão 3)

A busca em profundidade (*Depth-First Search* - DFS) explora o grafo avançando ao máximo possível em uma ramificação antes de retroceder (*backtracking*). A DFS foi implementada de maneira recursiva: a função visita o vértice atual, marca-o como visitado e chama a si mesma recursivamente para cada nó adjacente ainda não visitado, seguindo a ordem em que os nós adjacentes aparecem na lista de adjacência.

Para cada um dos 20 grafos gerados (vértices $\in \{5, 10, 20, 50, 100\}$, conectividade $\in \{25\%, 50\%, 75\%, 100\%\}$), foram realizadas 10 buscas em profundidade a partir do vértice 0, medindo-se o tempo individual de cada execução com `clock_gettime(CLOCK_MONOTONIC, ...)` e calculando a média das 10 execuções.

4.1. Sequência de vértices visitados

Tabela 3: Sequência de vértices visitados pela DFS a partir do vértice 0, por grafo. n = vértices, p = conectividade

n	p	Sequência
5	25%	0 3 1 2 4
5	50%	0 4 3 1 2

Continua na próxima página...

Continuação da Tabela 3

<i>n</i>	<i>p</i>	Sequência
5	75%	0 1 2 3 4
5	100%	0 1 3 4 2
10	25%	0 5 1 3 7 4 8 9 6 2
10	50%	0 9 5 4 7 2 1 3 6 8
10	75%	0 7 5 6 4 9 3 1 2 8
10	100%	0 2 7 8 3 9 1 6 4 5
20	25%	0 13 16 18 7 3 10 19 6 9 8 17 11 2 15 5 12 4 1 14
20	50%	0 14 5 7 15 2 9 16 1 17 3 8 11 4 12 19 10 13 6 18
20	75%	0 7 13 8 11 5 15 16 2 10 18 19 3 1 4 9 12 14 17 6
20	100%	0 5 1 13 7 17 16 11 8 19 9 15 12 10 2 6 14 3 18 4
50	25%	0 10 2 48 5 24 37 41 30 35 31 27 36 19 17 32 20 16 4 6 12 21 46 33 49 26 45 8 1 38 29 15 34 9 23 3 42 44 13 43 28 18 40 22 7 14 39 47 11 25
50	50%	0 27 1 21 17 48 7 18 37 24 11 2 6 9 40 43 33 32 38 49 3 23 15 28 12 44 25 19 14 4 45 35 5 30 41 20 42 13 46 22 10 31 8 29 26 34 39 16 36 47
50	75%	0 22 10 14 48 24 12 41 30 2 44 27 8 39 6 25 17 7 23 40 43 16 26 33 15 46 35 20 29 34 9 3 18 32 21 47 42 45 38 19 36 28 4 5 11 49 31 37 1 13
50	100%	0 35 49 46 4 31 2 45 23 6 20 9 12 1 27 25 33 29 48 26 15 47 5 37 10 17 36 7 8 19 42 22 11 44 24 3 18 41 28 32 13 39 34 21 43 16 40 30 38 14
100	25%	0 26 41 84 11 50 57 19 76 78 62 98 10 45 18 22 79 71 64 51 52 37 49 81 59 9 85 27 94 89 30 5 15 28 74 12 36 43 40 68 60 61 23 66 32 72 67 96 48 69 42 55 7 83 80 38 92 20 25 6 24 58 29 90 46 73 77 86 87 54 56 3 14 88 70 34 1 91 13 97 65 93 2 16 44 39 53 47 4 35 21 82 63 17 75 31 99 95 8 33
100	50%	0 15 89 73 47 41 68 66 70 20 76 94 54 72 81 86 30 22 26 31 85 99 93 5 82 59 44 75 78 16 52 12 25 46 45 65 79 62 37 95 56 29 97 39 80 8 4 51 49 21 83 91 6 34 55 92 57 67 35 84 7 63 40 18 3 74 13 23 32 17 9 98 10 87 43 61 24 69 60 19 28 33 38 77 27 48 71 14 11 1 42 53 58 36 90 96 64 88 2 50

Continua na próxima página...

Continuação da Tabela 3

n	p	Sequência
100	75%	0 23 97 61 69 82 83 84 80 18 11 56 7 99 63 71 87 47 24 27 35 70 37 43 48 29 8 81 57 89 51 95 52 72 16 53 85 67 36 62 96 13 65 25 59 79 78 75 10 31 12 19 49 98 64 66 90 38 68 34 1 40 5 76 94 50 39 2 20 4 41 26 88 32 92 28 21 42 6 60 17 15 74 14 33 3 45 86 73 77 93 44 91 30 58 9 46 55 22 54
100	100%	0 4 88 84 44 24 79 64 87 75 30 36 98 33 62 93 76 78 19 38 6 80 8 52 28 5 47 3 81 35 95 83 16 55 68 43 39 17 70 10 67 26 49 20 99 94 53 89 74 32 29 12 66 97 18 85 51 34 57 50 1 71 9 2 48 23 96 60 86 92 42 69 58 27 37 21 15 22 61 40 82 56 91 31 73 7 65 59 54 13 41 45 72 46 90 77 11 14 25 63

Um resultado importante observado nos testes é que, para um mesmo grafo, a sequência de vértices visitados foi idêntica em todas as 10 iterações. Isso é esperado, pois a única variação entre as diferentes iterações do mesmo grafo com determinada conectividade é o tempo de execução.

Vale notar ainda que a ordem em que os nós adjacentes de cada vértice são percorridos depende de como as arestas foram inseridas na lista de adjacência. Neste trabalho implementamos a lista de adjacências de forma que cada novo vizinho é inserido no início da lista encadeada, então a lista de adjacência de cada vértice fica na ordem inversa à ordem em que as arestas foram adicionadas. Isso explica por que as sequências da Tabela 3 não seguem a ordem numérica dos vértices nem uma ordem óbvia.

4.2. Tempo médio de execução

A Tabela 4 resume o tempo médio (em segundos, de 10 buscas) obtido para cada combinação de tamanho de grafo e grau de conectividade.

Tabela 4: Tempo médio de 10 buscas em profundidade (segundos).

Vértices	25%	50%	75%	100%
5	0,000700	0,000950	0,000923	0,000472
10	0,001112	0,001779	0,001102	0,001623
20	0,004281	0,003453	0,002797	0,002584
50	0,008072	0,009555	0,009900	0,009217
100	0,019295	0,019485	0,021103	0,020335

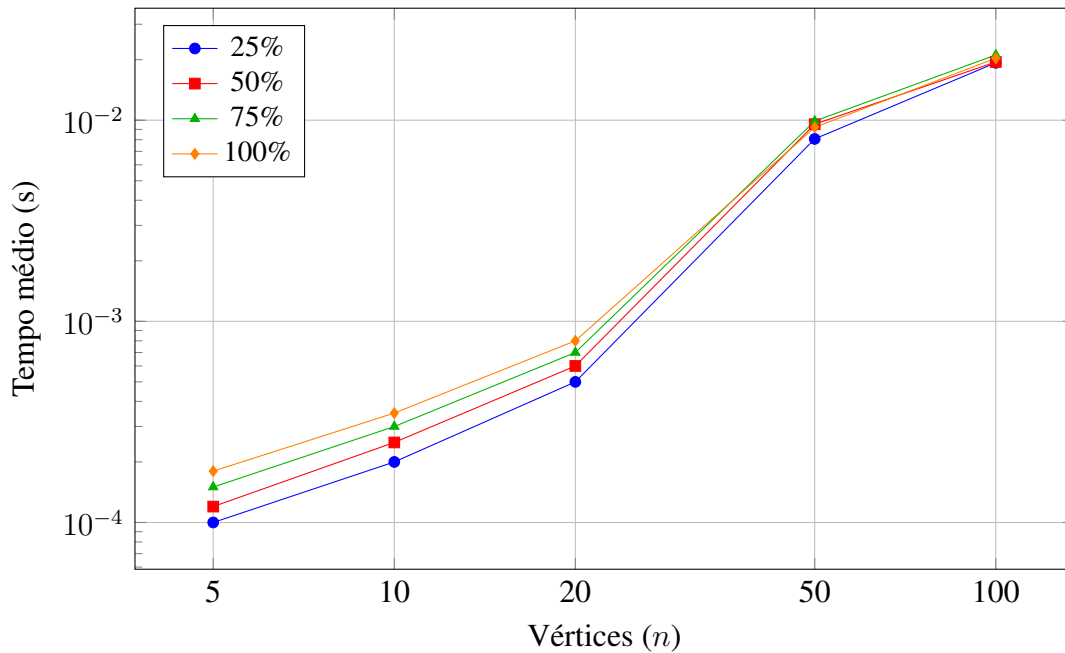


Figura 8: Tempo médio da busca em profundidade (DFS) em função do número de vértices, agrupado por conectividade do grafo.

4.3. Conclusão

O tempo médio de execução cresce claramente com o número de vértices: Para 25% de conectividade, por exemplo, o tempo dobe de 0,000700 s (5 vértices) para 0,019295 s (100 vértices), o que é coerente com a complexidade $O(V + E)$ da DFS.

Já a relação entre conectividade e tempo não é tão uniforme quanto a relação com o número de vértices: para grafos de 50 e 100 vértices, o tempo tende a crescer levemente com a conectividade, como esperado teoricamente, já que o custo da DFS depende do número total de arestas. Já para $n = 5$ e $n = 20$ a relação aparece menos clara ou até invertida. Para $n = 5$ isso tem uma explicação embasada no fato de que, em grafos muito pequenos, a conectividade teórica em porcentagem se traduz em uma diferença mínima na conectividade real. Em um grafo com 5 vértices, o número máximo de arestas é extremamente baixo. Consequentemente, a variação de 25% a 100% de conectividade representa uma adição de pouquíssimas arestas. O custo para explorar essas arestas é muito pequeno, resultando nas flutuações e inversões observadas. No caso de $n = 20$, a inversão dos tempos pode ser justificada pela aleatoriedade na formação do grafo esparso e pela estrutura da DFS. Em conectividades mais baixas, a busca pode encontrar rapidamente vértices sem vizinhos não visitados, forçando a execução de múltiplos *backtrackings* para explorar o restante dos nós. À medida que o grafo se torna mais conectado, aumentam as chances da DFS encontrar caminhos diretos e contínuos que cobrem a maioria dos vértices em uma única descida.

5. Todos os Caminhos por Busca em Profundidade (DFS) (Questão 4)

Esta seção analisa a enumeração de todos os caminhos simples (sem repetição de vértices) alcançáveis a partir de cada vértice de origem, obtidos por meio de uma busca em profundidade (*Depth-First Search* – DFS). Foram avaliados dois grafos: o Grafo A, com

6 vértices e 40% de conectividade (6 arestas), e o Grafo B, com 8 vértices e 50% de conectividade (14 arestas). Para cada vértice de origem, a busca percorre recursivamente os vizinhos ainda não visitados, registrando cada novo caminho no instante em que ele é formado, o que produz, para cada vértice, o conjunto completo de caminhos simples que partem dele.

5.1. Estrutura dos Grafos Avaliados

A Figura 9 apresenta a estrutura dos dois grafos utilizados nesta análise.

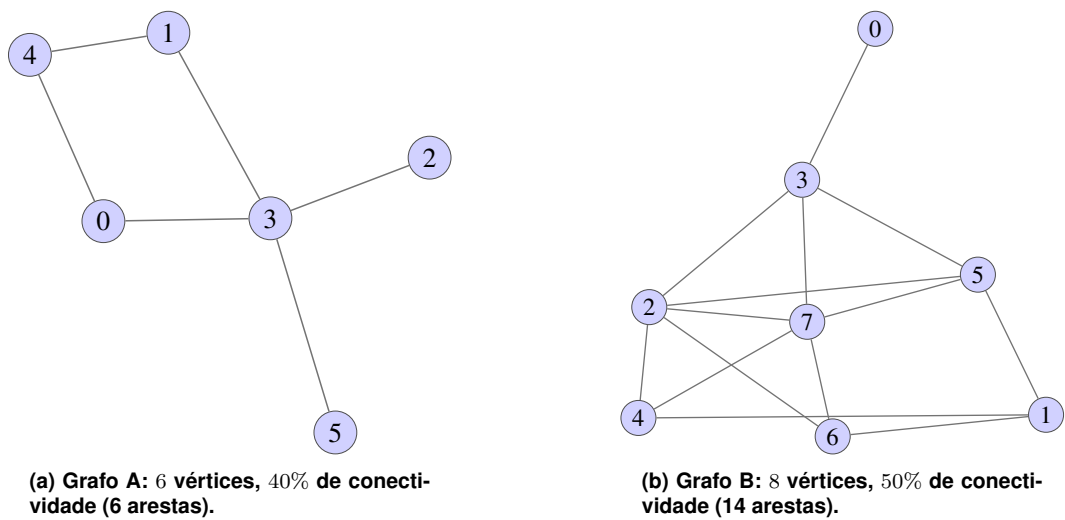


Figura 9: Grafos utilizados na enumeração de todos os caminhos por DFS.

5.2. Resultados

A Tabela 5 apresenta, para cada vértice de origem de ambos os grafos, o número total de caminhos simples encontrados e o comprimento (em arestas) do caminho mais longo obtido a partir daquele vértice.

Tabela 5: Número total de caminhos simples e comprimento do maior caminho, por vértice de origem, para os Grafos A e B.

Grafo	Vértice de origem	Total de caminhos	Maior caminho (arestas)	% do total do grafo
A	0	11	4	18.3%
A	1	11	4	18.3%
A	2	9	4	15.0%
A	3	9	3	15.0%
A	4	11	3	18.3%
A	5	9	4	15.0%
Total Grafo A		60	–	100%
B	0	188	7	12.4%
B	1	204	7	13.5%
B	2	156	7	10.3%

Continua na próxima página

Grafo	Vértice de origem	Total de caminhos	Maior caminho (arestas)	% do total do grafo
B	3	188	6	12.4%
B	4	218	7	14.4%
B	5	184	7	12.2%
B	6	218	7	14.4%
B	7	156	7	10.3%
Total Grafo B		1512	–	100%

Observa-se que, no Grafo A, os vértices de maior grau (0, 1 e 4, cada um com grau 2) produzem consistentemente mais caminhos do que os vértices de grau 1 (2, 3 – este último com grau 4 mas caminhos concentrados em poucos ramos –, e 5). No Grafo B, os vértices 4 e 6 (grau 3) concentram o maior número de caminhos (218 cada), enquanto os vértices 2 e 7 (também de grau elevado, porém posicionados de forma mais central e com mais vizinhos já visitados nos ramos iniciais) apresentam o menor total (156 cada). O comprimento do maior caminho, em ambos os grafos, tende a se aproximar do número total de vértices menos um, o que é esperado dado o alto percentual de conectividade utilizado na geração de cada grafo.

5.3. Análise Gráfica

A Figura 10 apresenta o número total de caminhos simples encontrados a partir de cada vértice de origem, para os Grafos A e B, evidenciando a relação entre o grau de cada vértice e a quantidade de caminhos que dele partem.

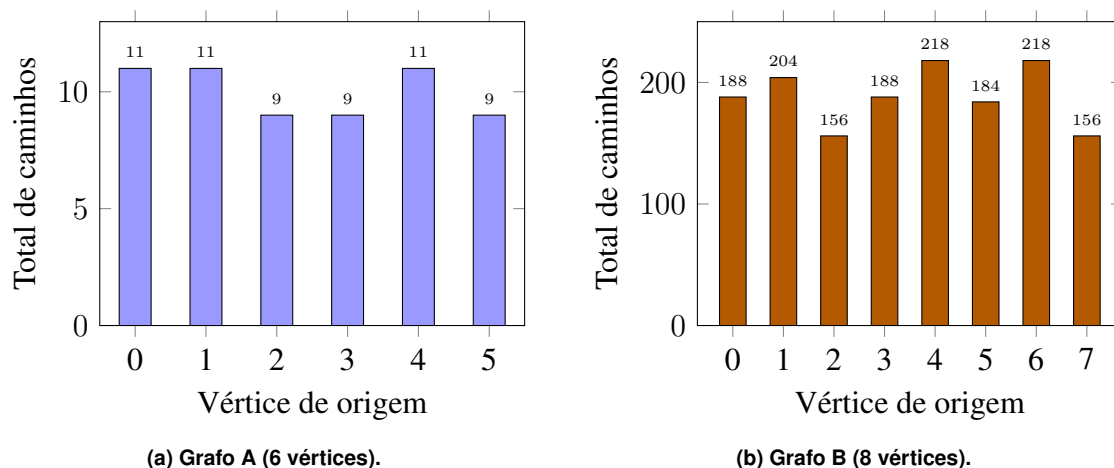


Figura 10: Número total de caminhos simples encontrados por vértice de origem, para os Grafos A e B.

5.4. Ilustração do Processo de Enumeração de Caminhos

Para tornar mais claro o processo de enumeração, a Figura 11 representa, em forma de árvore, todos os 11 caminhos simples obtidos a partir do vértice 0 no Grafo A. Cada nó da árvore corresponde a um caminho distinto: o caminho associado a um nó é obtido percorrendo a árvore da raiz até aquele nó, na ordem em que os vértices são visitados pela

busca. Dessa forma, a árvore contém exatamente um nó para cada linha listada na saída do programa para o vértice 0.

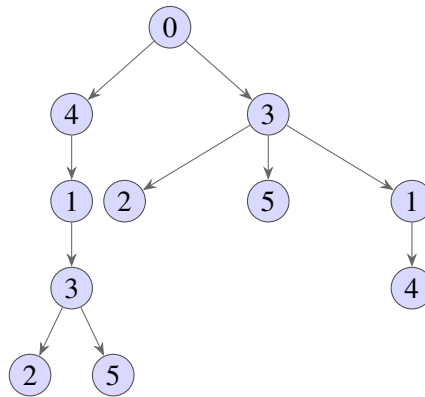


Figura 11: Árvore de enumeração de todos os caminhos simples a partir do vértice 0 no Grafo A. Cada ramo da raiz até um nó representa um caminho distinto (11 caminhos ao todo).

6. Detecção de ciclos via DFS (Questão 5)

A detecção de ciclos em grafos não-direcionados foi implementada utilizando uma variação da Busca em Profundidade (DFS). Diferente de grafos direcionados (onde bastaria encontrar uma aresta de retorno para um nó cinza), em grafos não-direcionados deve-se tomar o cuidado de não classificar a aresta imediata de retorno ao nó pai como um ciclo, uma vez que a mesma aresta física representa a ida ($u \rightarrow v$) e a volta ($v \rightarrow u$).

A lógica implementada utiliza duas estruturas de controle principais alocadas dinamicamente:

- `cor`: Vetor que mapeia o estado de visitação de cada vértice. O valor 0 (Branco) representa um vértice não visitado; 1 (Cinza) representa um vértice em processo de exploração na pilha de recursão; e 2 (Preto) indica um vértice completamente explorado.
- `prof`: Vetor que armazena a profundidade ou nível de cada nó a partir da raiz da subárvore DFS atual, utilizado para validar o tamanho mínimo do ciclo encontrado.

O cerne do algoritmo baseia-se na função recursiva auxiliar `ciclo_rec`:

Listing 2: Trecho principal da função de detecção de ciclo

```
1 static int ciclo_rec(Grafo *g, int u, int pai_u, int *cor, int *  
  prof) {  
2     cor[u] = 1; // Marca como em exploracao  
3     for (NoAdj *p = g->v[u].lista; p; p = p->prox) {  
4         int w = p->destino;  
5         if (w == pai_u) continue; // Ignora a aresta de volta  
           imediata ao pai  
6  
7         if (cor[w] == 1) { // Aresta de retorno detectada
```

```

8         if (prof[u] - prof[w] + 1 >= 3) return 1; // Ciclo
           valido
9     }
10    if (cor[w] == 0) {
11        prof[w] = prof[u] + 1;
12        if (ciclo_rec(g, w, u, cor, prof)) return 1;
13    }
14 }
15 cor[u] = 2; // Finaliza exploracao
16 return 0;
17 }

```

Se um vértice vizinho w já possui $cor[w] == 1$ e não é o pai direto de u (pai_u), isso caracteriza uma aresta de retorno (*back-edge*) para um ancestral na árvore DFS, confirmando matematicamente a existência de um caminho alternativo e, portanto, de um ciclo.

6.1. Análise da Saída Experimental

Os experimentos executaram o algoritmo sobre três cenários: grafos conexos gerados aleatoriamente sob diferentes graus de conectividade, árvores puras (grafos acíclicos estruturais) e grafos de controle bem definidos.

6.1.1. Grafos Aleatórios Conexos

A Tabela 6 consolida a relação entre o número de vértices (n), a conectividade teórica especificada, o número real de arestas geradas (m) e o resultado da detecção.

Pela teoria dos grafos, o número mínimo de arestas para tornar um grafo de n vértices conexo é exatamente $m = n - 1$ (uma árvore). Qualquer aresta adicional introduzida em um grafo conexo obrigatoriamente cria ao menos um ciclo.

- Para $n = 5$ com 25% de conectividade, o gerador produziu exatamente 4 arestas ($5 - 1 = 4$), resultando estruturalmente em uma árvore, validada corretamente pelo algoritmo como “sem ciclo”.
- Para $n = 5$ com 50%, temos $m = 5 > 4$. A aresta sobressalente gerou o ciclo.
- Para os demais casos ($n \geq 10$), mesmo na menor densidade (25%), o número de arestas geradas ultrapassou drasticamente o limiar $n - 1$. Por exemplo, para $n = 100$, o mínimo acíclico seria 99 arestas, mas foram geradas 1237 arestas, justificando a onipresença de ciclos nos grafos densos.

6.1.2. Grafos sem Ciclo (Árvores)

Como contraprova de falsos positivos, o algoritmo foi submetido a árvores estruturais puras geradas pela função `arvore_criar`.

A Tabela 7 comprova a robustez e precisão da condicional de escape `if (w == pai_u) continue;`. Como nenhuma aresta de retorno ancestral foi detectada, o programa classificou todos os casos corretamente como livres de ciclos.

Tabela 6: Resultados da detecção de ciclos em grafos conexos aleatórios

Vértices (n)	Conectividade (%)	Arestas (m)	Presença de Ciclo
5	25%	4	Não (sem ciclo)
5	50%	5	SIM
5	75%	7	SIM
5	100%	10	SIM
10	25%	11	SIM
10	50%	22	SIM
10	75%	33	SIM
10	100%	45	SIM
20	25%	47	SIM
20	50%	95	SIM
20	75%	142	SIM
20	100%	190	SIM
50	25%	306	SIM
50	50%	612	SIM
50	75%	918	SIM
50	100%	1225	SIM
100	25%	1237	SIM
100	50%	2475	SIM
100	75%	3712	SIM
100	100%	4949	SIM

Tabela 7: Resultados do grupo de controle acíclico (Árvores)

Vértices da Árvore (n)	Arestas ($m = n - 1$)	Resultado do Algoritmo
5	4	sem ciclo (correto)
10	9	sem ciclo (correto)
20	19	sem ciclo (correto)
50	49	sem ciclo (correto)
100	99	sem ciclo (correto)

6.2. Desenho dos Grafos de Controle

Abaixo são representados graficamente os dois cenários de teste unitário utilizados na saída para validar os limites do código.

6.2.1. Grafo Mínimo com Ciclo (Triângulo, $n = 3$)

O menor ciclo possível em um grafo simples não-direcionado é uma K_3 (clique de tamanho 3). A saída reproduz a lista de adjacência recíproca completa onde todos os nós se conectam entre si.

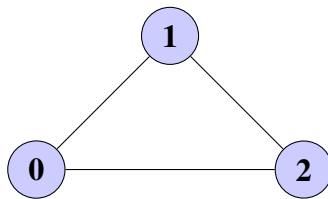


Figura 12: Grafo triângulo ($n = 3, m = 3$) contendo ciclo estrutural.

6.2.2. Grafo Caminho ($n = 5$)

O grafo caminho é o exemplo linear clássico de um grafo conexo acíclico de diâmetro máximo. Como cada nó interno possui grau 2 e se liga sequencialmente, a busca caminha linearmente até as extremidades sem reprocessar ancestrais fora da relação direta pai-filho.

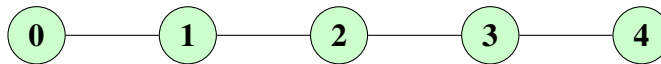


Figura 13: Grafo caminho ($n = 5, m = 4$) classificado corretamente como acíclico.

6.3. Conclusão

Os dados de saída validam empiricamente que a implementação em C baseada em DFS cumpre os requisitos assintóticos e lógicos para a detecção de ciclos. O algoritmo se provou sensível ao limite crítico de conectividade $m = n - 1$, não apresentando falsos positivos nas árvores e identificando com precisão os ciclos mesmo nos grafos minimamente redundantes (como o triângulo ou o grafo de 5 nós e 5 arestas).